A Tableau Calculus for $\mathcal{ALCI}_{RCC5}$ with Tri-Neighborhood Blocking

Third revision, April 2026

Claude (Anthropic) Michael Wessel

April 2026

Status (April 2026). This paper proves **completeness** of the Tri-neighborhood tableau calculus. **Termination** is **disproved** (Section 4.1). **Soundness** has a **formal gap**: Lemma 5.8 (non-empty domains after arc-consistency) is supported by computational verification on completion graphs of size 8, 10, and 12, but lacks a general formal proof. This is structurally analogous to the extension gap in the companion paper [4]: a disjunctive constraint network must be shown solvable, and no general argument is provided. The calculus is therefore **not a proven decision procedure**: termination fails and soundness is incomplete. Completeness is the only fully established component.

Abstract

We present a tableau-based decision procedure for concept satisfiability in $\mathcal{ALCI}_{RCC5}$, the description logic $\mathcal{ALCI}$ extended with RCC5 composition-based role interactions over a complete relational structure. The key innovation is *Tri-neighborhood blocking*: a node x is blocked by an earlier node y only when they share the same concept label, the same set of abstract triangle types—tuples $(\tau_1, R_{12}, \tau_2, R_{23}, \tau_3, R_{13})$ recording Hintikka types and RCC5 relations but no node identities—and, for each (relation, type) pair, the same distribution of triangle-type sets among their respective neighbors. We prove completeness (a model guides all nondeterministic choices) and present a soundness argument (model construction by triangle-type-filtered unraveling, with global consistency from the full tractability of RCC5). **However, the soundness proof has a formal gap**: the key lemma (non-empty domains after arc-consistency) is supported by computational verification but not by a general formal argument. However, **termination fails**: a computational implementation reveals that the concept $A \sqcap \exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.(\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B)$ causes unbounded node creation. The mechanism is not blocking/unblocking oscillation (zero unblocking events occur) but *frontier advancement*: the newest node in a PP-chain always lacks PP-successor triangle types ($|\mathsf{Tri}| = 8$) compared to interior nodes ($|\mathsf{Tri}| = 16$), so it is never blocked. This confirms the "blocking dilemma": type-equality blocking terminates but risks unsound unraveling, while Tri-neighborhood blocking ensures sound unraveling but does not terminate. The dilemma remains open.

1 Introduction

The description logic $\mathcal{ALCI}_{RCC5}$ extends $\mathcal{ALCI}$ with a role box defined by the RCC5 composition table: the five base relations $\{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ act as role names, and every pair of domain elements is related by exactly one base relation, subject to global composition consistency [1].

Models are thus complete graphs with composition-consistent edge labels—a fundamental departure from tree-shaped $\mathcal{ALCI}$ models.

The decidability of $\mathcal{ALCI}_{RCC5}$ has remained open since Wessel [1] established decidability of $\mathcal{ALCI}_{RCC3}$ and proved $\mathcal{ALCI}_{RCC5}$ PSPACE-hard. Several approaches have been attempted: a quasimodel method with type elimination [4], a contextual tableau with local states [5], direct model construction [6], and a two-tier quotient [7] that proves decidability for the *PO-coherent* fragment.

All approaches encounter the same structural obstacle: the complete-graph semantics require assigning globally consistent RCC5 relations across an infinite model. The *patchwork property* of RCC5 [2]—that path-consistent atomic networks are globally consistent—bridges local to global consistency, but the question is whether the finite information in a tableau completion graph suffices to guarantee path-consistency of the infinite unraveled structure.

The blocking dilemma

In a standard DL tableau, blocking ensures termination: a blocked node x copies the witness structure of its blocker y during model extraction. In complete-graph models, this copying ("unraveling") must preserve RCC5 composition consistency across *all* triples, not just the parent-child edge.

This creates a dilemma, identified in [7]:

- **Type-equality blocking** (block when $\mathcal{L}(x) = \mathcal{L}(y)$) always terminates but may produce *novel triangle types* during unraveling—triples $(\tau_1, R, \tau_2, S, \tau_3, R')$ not witnessed in the completion graph—potentially violating path-consistency.
- **Node-identity profile blocking** (block when x and y have identical relational profiles to all other nodes) always supports correct unraveling but may *never terminate*: for PO-incoherent descriptors, the "sliding PO diagonal" gives each node a unique profile.

Tri-neighborhood blocking

We *attempted* to resolve this dilemma by observing that the non-termination argument for node-identity blocking uses *concrete* node identities, while the correctness argument only needs *abstract* relational patterns. An *abstract triangle type* is a tuple $(\tau_1, R_{12}, \tau_2, R_{23}, \tau_3, R_{13})$ recording three Hintikka types and three pairwise RCC5 relations—no node identities. The *triangle-type set* of a node x is the set of all abstract triangle types that x participates in.

Computational verification on the PO-incoherent counterexample (a 24-element PP-chain with 12 PO-witnesses) shows that abstract triangle-type sets *stabilize for interior nodes*: all interior τ_A nodes share exactly the same 68 abstract triangle types, despite having different concrete neighbors.

We define *Tri-neighborhood blocking*: in addition to requiring $\mathcal{L}(x) = \mathcal{L}(y)$ and $\mathsf{Tri}(x) = \mathsf{Tri}(y)$, we require that for each (relation, type) pair, the set of Tri-values among the respective neighbors of x and y also matches. This ensures the copy is faithful not just from x/y's perspective, but from *every neighboring node's perspective*.

Tri-neighborhood blocking is *intended* to achieve:

1. **Correct unraveling**: copying y's witness structure for x produces only triangle types already in the completion graph's set T, verified from every participating node's perspective.
2. **Global consistency**: T-closure plus full RCC5 tractability [2, 3] yields a globally consistent atomic network.

However, the formal proof of item 2 is incomplete: the non-emptiness of constraint-network domains after arc-consistency (Lemma 5.8) relies on computational verification rather than a general argument. However, a full implementation [9] reveals that **termination fails**: frontier nodes in a chain always have a smaller Tri than interior nodes (they lack successor-edge triangles), so they are never blocked. The blocking dilemma remains open (Section 4.1).

2 Preliminaries

2.1 The logic $\mathcal{ALCI}_{RCC5}$

Concepts are built from concept names, the RCC5 base relations $R \in \{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ as role names, and the constructors $\sqcap$, $\sqcup$, $\neg$, $\exists R.C$, $\forall R.C$ in negation normal form (NNF). We adopt **strong EQ semantics**: $\rho(d, e) = \mathsf{EQ}$ iff $d = e$.

An interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ assigns a total function $\rho^{\mathcal{I}} : \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}} \to \{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ subject to: (i) $\rho(d, d) = \mathsf{EQ}$; (ii) $\rho(d, e) = \mathsf{inv}(\rho(e, d))$; (iii) composition consistency: $\rho(d_1, d_3) \in \mathsf{comp}(\rho(d_1, d_2), \rho(d_2, d_3))$ for all d_1, d_2, d_3; and the usual concept-interpretation clauses for $\mathcal{ALCI}$.

2.2 Types, closure, composition table

Given C_0 in NNF, $\mathsf{cl}(C_0)$ is the Fischer–Ladner closure. A *Hintikka type* $\tau \subseteq \mathsf{cl}(C_0)$ is a maximal propositionally consistent subset. Write $\mathsf{Tp}(C_0)$ for the set of types; $|\mathsf{Tp}(C_0)| \leq 2^{|\mathsf{cl}(C_0)|}$.

Write $\mathsf{NR}^- = \{\mathsf{DR}, \mathsf{PO}, \mathsf{PP}, \mathsf{PPI}\}$ for the base relations between distinct elements (EQ excluded).

The composition table $\mathsf{comp} : (\mathsf{NR}^-)^2 \to 2^{\mathsf{NR}^-}$ is:

$\mathsf{comp}(R, S)$	DR	PO	PP	PPI
DR	{all}	{DR,PO,PP}	{DR,PO,PP}	{DR}
PO	{DR,PO,PPI}	{all}	{PO,PP}	{DR,PO,PPI}
PP	{DR}	{DR,PO,PP}	{PP}	{all}
PPI	{DR,PO,PPI}	{PO,PPI}	{PO,PP,PPI}	{PPI}

2.3 Type-safety

Definition 2.1 (Type-safe relations)**.** For Hintikka types τ_1, τ_2, define

$$\mathsf{Safe}(\tau_1, \tau_2) = \big\{R \in \mathsf{NR}^- : \forall\, (\forall R.C) \in \tau_1,\ C \in \tau_2;\ \forall\, (\forall \mathsf{inv}(R).C) \in \tau_2,\ C \in \tau_1\big\}.$$

In any model, $\rho(d, e) \in \mathsf{Safe}(\mathsf{tp}(d), \mathsf{tp}(e))$ for distinct d, e.

2.4 Full tractability of RCC5

Theorem 2.2 (Renz & Nebel [2], Renz [3])**.** *Every path-consistent disjunctive RCC5 constraint network over* $\{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ *is satisfiable. Equivalently: if arc-consistency enforcement does not empty any domain, a consistent atomic refinement exists.* □

This is the *patchwork property*: local consistency (path-consistency) implies global consistency.

3 The Tableau Calculus

3.1 Completion graphs

Definition 3.1 (Completion graph)**.** A *completion graph* for C_0 is a tuple $\mathcal{G} = (V, \mathcal{L}, \mathcal{E}, \prec)$ where:

- V is a finite non-empty set of nodes.
- $\mathcal{L} : V \to 2^{\mathsf{cl}(C_0)}$ is a labelling function.
- $\mathcal{E} : \{(x, y) \in V^2 \mid x \neq y\} \to \mathsf{NR}^-$ assigns a base relation to every ordered pair of distinct nodes, with $\mathcal{E}(y, x) = \mathsf{inv}(\mathcal{E}(x, y))$.
- $\prec$ is a strict total order on V (creation order).

The *initial* completion graph has a single node x_0 with $\mathcal{L}(x_0) = \{C_0\}$.

3.2 Triangle types

Definition 3.2 (Abstract triangle type)**.** An *abstract triangle type* is a tuple $(\tau_1, R_{12}, \tau_2, R_{23}, \tau_3, R_{13})$ where $\tau_1, \tau_2, \tau_3 \in \mathsf{Tp}(C_0)$ and $R_{12}, R_{23}, R_{13} \in \mathsf{NR}^-$ with $R_{13} \in \mathsf{comp}(R_{12}, R_{23})$.

Definition 3.3 (Triangle-type set of a node)**.** For a node x in $\mathcal{G}$, the *triangle-type set* of x is:

$$\mathsf{Tri}(x) = \big\{(\mathcal{L}(x), \mathcal{E}(x, b), \mathcal{L}(b), \mathcal{E}(b, c), \mathcal{L}(c), \mathcal{E}(x, c)) : b, c \in V,\ |\{x, b, c\}| = 3\big\}.$$

This is the set of all abstract triangle types in which x participates (at the first position).

Remark 3.4. Since each abstract triangle type uses only Hintikka types and RCC5 relations—no node identities—the set $\mathsf{Tri}(x)$ is drawn from the finite universe $\mathsf{Tp}(C_0)^3 \times (\mathsf{NR}^-)^3$. Its cardinality is bounded by $|\mathsf{Tp}|^2 \cdot |\mathsf{NR}^-|^2 = 2^{2|\mathsf{cl}(C_0)|} \cdot 16$ (since the first component $\tau_1 = \mathcal{L}(x)$ is fixed).

Definition 3.5 (Triangle-type set of the graph)**.** The *triangle-type set* of $\mathcal{G}$ is:

$$T(\mathcal{G}) = \big\{(\mathcal{L}(x), \mathcal{E}(x, y), \mathcal{L}(y), \mathcal{E}(y, z), \mathcal{L}(z), \mathcal{E}(x, z)) : \text{distinct } x, y, z \in V\big\}.$$

3.3 Tri-neighborhood signature

Definition 3.6 (Tri-neighborhood signature)**.** For a node x in $\mathcal{G}$, the *Tri-neighborhood signature* of x is:

$$\mathsf{TNbr}(x) = \big\{\big((R, \tau),\ \{\mathsf{Tri}(b) : b \in V,\ b \neq x,\ \mathcal{E}(x, b) = R,\ \mathcal{L}(b) = \tau\}\big) : R \in \mathsf{NR}^-,\ \tau \in \mathsf{Tp}\big\}.$$

That is, for each (relation, type) pair (R, τ), $\mathsf{TNbr}(x)$ records the *set* of distinct Tri-values occurring among x's R-neighbors of type τ.

Remark 3.7. The Tri-neighborhood signature captures not only which abstract triangle types x participates in, but also which abstract triangle types each of x's neighbors participates in, grouped by how the neighbor relates to x. This is analogous to *pairwise blocking* in $\mathcal{ALCI}$ tree tableaux (which matches a node *and* its parent with the blocker and its parent), generalized to the complete-graph setting: instead of matching one parent edge, we match *all* neighbor edges grouped by (relation, type).

3.4 Blocking

Definition 3.8 (Tri-neighborhood blocking)**.** A node $x \in V$ is *blocked* if there exists $y \prec x$ such that y is not blocked and:

(i) $\mathcal{L}(x) = \mathcal{L}(y)$ (same concept label),
(ii) $\mathsf{Tri}(x) = \mathsf{Tri}(y)$ (same triangle-type set), and
(iii) $\mathsf{TNbr}(x) = \mathsf{TNbr}(y)$ (same Tri-neighborhood signature).

We call y a *blocker* for x. A node is *active* if it is not blocked.

Remark 3.9. Conditions (ii) and (iii) are the key innovations. Simple type-equality blocking (condition (i) alone) always terminates but may produce novel triangle types during unraveling. Condition (ii) ensures that x and y participate in the same abstract relational patterns. Condition (iii) additionally ensures that the *neighbors* of x and y also have matching triangle-type sets: for each (relation, type) pair, the menu of Tri-values among neighbors is identical.

This strengthening is crucial for the soundness proof: when we copy y's witness structure for blocked node x, we need the triangle types to be in T not just from x/y's perspective, but from *every neighbor's perspective*. Condition (iii) guarantees this.

Note that conditions (ii)–(iii) do *not* require x and y to be related to the same concrete nodes. They require only that the *abstract neighborhood patterns* match. This is strictly weaker than node-identity profile matching, which is why the condition still terminates (Section 4).

3.5 Expansion rules

The calculus has four deterministic rules and two nondeterministic rules.

Propositional rules (deterministic):

($\sqcap$) If $D_1 \sqcap D_2 \in \mathcal{L}(x)$ and $\{D_1, D_2\} \not\subseteq \mathcal{L}(x)$: add D_1 and D_2 to $\mathcal{L}(x)$.

($\sqcup$) If $D_1 \sqcup D_2 \in \mathcal{L}(x)$ and $D_1 \notin \mathcal{L}(x)$ and $D_2 \notin \mathcal{L}(x)$: *nondeterministically* add D_1 or D_2.

Propagation rule (deterministic):

($\forall$) If $\forall R.D \in \mathcal{L}(x)$ and $\mathcal{E}(x, y) = R$ and $D \notin \mathcal{L}(y)$: add D to $\mathcal{L}(y)$.

This handles inverses: $\forall R^-.D$ becomes $\forall \mathsf{inv}(R).D$ in NNF.

Generating rule (nondeterministic):

($\exists$) If $\exists R.D \in \mathcal{L}(x)$, x is active, and no $y \in V$ satisfies $\mathcal{E}(x, y) = R$ and $D \in \mathcal{L}(y)$:

1. Create a fresh node y, $\prec$-greatest.
2. Set $\mathcal{L}(y) := \{D\}$.
3. Set $\mathcal{E}(x, y) := R$, $\mathcal{E}(y, x) := \mathsf{inv}(R)$.
4. For each $z \in V \setminus \{x, y\}$, *nondeterministically* choose $S_z \in \mathsf{NR}^-$ and set $\mathcal{E}(z, y) := S_z$, $\mathcal{E}(y, z) := \mathsf{inv}(S_z)$.

3.6 Constraint filtering

After step 4 of ($\exists$), we require:

(CF) Composition consistency. For every triple of distinct nodes (u, v, w) involving y:

$$\mathcal{E}(u, w) \in \mathsf{comp}(\mathcal{E}(u, v), \mathcal{E}(v, w)).$$

If no assignment of $\{S_z\}$ satisfies (CF) for all triples simultaneously, the branch *closes* (clash by composition failure).

Remark 3.10. Checking (CF) is an RCC5 constraint satisfaction problem. By Theorem 2.2, a simultaneous satisfying assignment exists iff path-consistency enforcement does not empty any domain. This can be checked in $O(|V|^3)$.

3.7 Rule application strategy

1. Apply ($\sqcap$), ($\sqcup$), ($\forall$) exhaustively to all nodes.
2. Evaluate blocking: x is blocked if $\mathcal{L}(x) = \mathcal{L}(y)$, $\mathsf{Tri}(x) = \mathsf{Tri}(y)$, and $\mathsf{TNbr}(x) = \mathsf{TNbr}(y)$ for some non-blocked $y \prec x$.
3. Apply one instance of ($\exists$) for an active node with an unsatisfied demand.
4. Apply ($\forall$) exhaustively (new edges may trigger propagation).
5. Re-evaluate blocking.
6. Repeat from step 1 until no rule applies or a clash is detected.

3.8 Clash conditions

Definition 3.11 (Saturated and clash-free)**.** $\mathcal{G}$ is *saturated* if no rule applies. $\mathcal{G}$ is *clash-free* if:

(C1) No x has $\{A, \neg A\} \subseteq \mathcal{L}(x)$.
(C2) Composition consistency (CF) holds for all triples.

A saturated, clash-free graph is *open*.

4 Termination

Theorem 4.1 (Non-termination)**.** *The Tri-neighborhood tableau does not terminate on all inputs. In particular, the concept*

$$C_0 = A \sqcap \exists \mathsf{PP}.A \sqcap \forall \mathsf{PP}.(\exists \mathsf{PP}.A \sqcap \exists \mathsf{DR}.B)$$

causes unbounded node creation.

The counterexample and its proof are given in Section 4.1 below. We first record the partial termination arguments that remain valid.

Valid partial bounds. Let $n = |\mathsf{cl}(C_0)|$ and $N_\tau = |\mathsf{Tp}(C_0)| \leq 2^n$.

Finiteness of blocking signatures. An abstract triangle type is an element of $\mathsf{Tp}^3 \times (\mathsf{NR}^-)^3$. Let S denote the number of possible Tri-values; $S \leq 2^{N_\tau^3 \cdot 64}$. A Tri-neighborhood signature assigns, to each of the at most $4 \cdot N_\tau$ pairs (R, τ), a subset of $\{1, \ldots, S\}$ (a set of Tri-values). The number of distinct TNbr-values is at most $(2^S)^{4N_\tau}$. The number of distinct $(\mathcal{L}, \mathsf{Tri}, \mathsf{TNbr})$ triples is therefore bounded by $A(C_0) := N_\tau \cdot S \cdot (2^S)^{4N_\tau}$.

Active node bound. Active nodes have pairwise distinct $(\mathcal{L}, \mathsf{Tri}, \mathsf{TNbr})$ triples (by the blocking definition). Hence the number of *simultaneously* active nodes is at most $A(C_0)$.

Labels grow monotonically. Rules only add concepts to labels, never remove them. Each label changes at most n times. Edge labels $\mathcal{E}(x, y)$ are set at node creation and never change.

Bounded branching. The ($\exists$)-rule for node x and demand $\exists R.D$ fires only when no witness exists. Once a witness y is created (with $\mathcal{E}(x, y) = R$, $D \in \mathcal{L}(y)$), the demand is *permanently* satisfied: D remains in $\mathcal{L}(y)$ (labels only grow) and $\mathcal{E}(x, y)$ never changes. Therefore each node fires ($\exists$) at most n times, creating at most n children in the creation tree.

Bounded total node creation. We must bound not just simultaneous active nodes but the *total* number of nodes ever created. (An earlier version of this proof appealed only to the active node bound, which is insufficient: a process with at most $A(C_0)$ active nodes could in principle cycle through blocking and unblocking, creating fresh successors each time.)

The correct argument has two parts.

Part 1: Permanent demand satisfaction limits re-expansion. When a previously blocked node x becomes unblocked (because its blocker's signature changed), x may fire ($\exists$) for its remaining unsatisfied demands. But demands satisfied in earlier active periods remain satisfied (labels and edges are permanent). So x fires ($\exists$) at most n times *total*, regardless of how many blocking/unblocking cycles it undergoes.

Part 2: Blocking/unblocking cycles are bounded per node. A node x transitions from blocked to active only when its own signature or its blocker's signature changes. Each such change requires either (a) a label growth at some node (at most n per node, irreversible), or (b) a $\mathsf{Tri}/\mathsf{TNbr}$ change caused by a new node creation or a label growth at a neighbor. In the *label-stable regime* (after all labels have reached their final values), $\mathsf{Tri}(x)$ can only grow monotonically: new nodes may add new triangle types, but no types are replaced (since the Hintikka-type component of each triangle is now fixed). Similarly, $\mathsf{TNbr}(x)$ can only grow. Since $\mathsf{Tri}(x) \subseteq T_{\max}$ (a finite set), $\mathsf{Tri}(x)$ changes at most $|T_{\max}|$ times in the label-stable regime. Combined with at most n label changes before the label-stable regime, each node's signature changes a bounded number of times.

Combining both parts: Each node has a bounded number of active periods (Part 2), and in each active period fires ($\exists$) for at most n demands (Part 1), each creating one child. By König's lemma, the creation tree—with bounded branching (at most n) and bounded active periods per node—is finite.

Subtlety. The argument above bounds Tri-changes per node in the label-stable regime, but Tri-changes *before* label stability involve replacement (not just growth) of triangle types when Hintikka types change. This pre-stable phase has at most n label changes per node, each causing bounded Tri-reconfiguration, so signature changes remain bounded but the precise bound is more complex. The total creation count is therefore bounded by a computable (non-elementary) function of $|C_0|$, but extracting a clean closed-form bound requires tracking the interaction between label growth and Tri-reconfiguration carefully. We revisit this in the honest assessment (Section 8).

Why these bounds are insufficient. The above establishes bounded branching, permanent demand satisfaction, and bounded signature changes per node in the label-stable regime. These local per-node bounds do *not* compose into a global finiteness argument (as noted by a reviewer [11]). The counterexample below shows that the gap is real: the calculus fails to terminate despite all local bounds being satisfied.

4.1 Counterexample: the frontier advancement problem

Proof of Theorem 4.1. Consider $C_0 = A \sqcap \exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.(\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B)$. This concept forces an infinite PP-chain where every chain node has type A and a DR-neighbor of type B. We show the tableau constructs such a chain without ever blocking the frontier node.

The completion graph. The tableau creates:

- n_0: the root, with $\mathcal{L}(n_0) = \{C_0, A, \exists\mathsf{PP}.A, \forall\mathsf{PP}.(\cdots), \top\}$.
- n_1: PP-successor of n_0, receiving $\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B$ via $\forall$-propagation from n_0.
- n_2: the sole DR-witness, with $\mathcal{L}(n_2) = \{B, \top\}$. By composition ($\mathsf{comp}(\mathsf{PP}, \mathsf{DR}) \supseteq \{\mathsf{DR}\}$), the edge from n_0 to n_2 is assigned DR.
- $n_3, n_4, n_5, \ldots$: further PP-chain nodes. Each n_k receives $\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B$ via $\forall$PP-propagation from n_0 (since $\mathcal{E}(n_0, n_k) = \mathsf{PP}$ by composition). Each n_k's $\exists\mathsf{DR}.B$ demand is satisfied by n_2 (since $\mathcal{E}(n_k, n_2) = \mathsf{DR}$ by composition). But $\exists\mathsf{PP}.A$ requires a PP-successor with A in its label.

Edge pattern. By the composition rule $\mathsf{comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$, every chain node n_k has:

- $\mathcal{E}(n_k, n_j) = \mathsf{PPI}$ for all $j < k$ (predecessors),
- $\mathcal{E}(n_k, n_j) = \mathsf{PP}$ for all $j > k$ (successors),
- $\mathcal{E}(n_k, n_2) = \mathsf{DR}$ (the witness).

The Tri asymmetry. For an interior node n_k ($k \geq 4$, with both predecessors *and* successors), $\mathsf{Tri}(n_k)$ contains triangle types involving PP-neighbors *and* PPI-neighbors:

$$|\mathsf{Tri}(n_k)| = 16.$$

For the frontier node n_f (the newest, with no successors), $\mathsf{Tri}(n_f)$ contains only triangle types involving PPI-neighbors and the DR-witness:

$$|\mathsf{Tri}(n_f)| = 8.$$

The 8 missing types all involve PP as an edge label—e.g., $(\tau_A, \mathsf{PP}, \tau_A, \mathsf{PP}, \tau_A, \mathsf{PP})$—which the frontier node lacks because it has no PP-edges. Since $\mathsf{PP} \neq \mathsf{PPI}$, these types cannot be obtained by relabeling PPI-triangles.

Non-termination. Since $\mathsf{Tri}(n_f) \subsetneq \mathsf{Tri}(n_k)$ for every interior node n_k, the blocking condition $\mathsf{Tri}(n_f) = \mathsf{Tri}(n_k)$ is never satisfied. The frontier node remains active, its $\exists\mathsf{PP}.A$ demand fires, creating n_{f+1}. The old frontier n_f gains a PP-successor and eventually reaches $|\mathsf{Tri}| = 16$, getting blocked by an earlier interior node. But n_{f+1} is now the new frontier with $|\mathsf{Tri}| = 8$, and the cycle repeats.

The active-node count stabilizes at approximately 9 (verified computationally up to 500 nodes), but total node creation is unbounded. Zero unblocking events occur: this is not oscillation but *frontier advancement.* □

Remark 4.2 (Scope of non-termination)**.** Computational testing [9] on 20 hand-crafted concepts and 200 random concepts shows that *every* concept requiring an infinite chain via $\forall$-propagation exhibits the same frontier-advancement pattern. The only concepts that terminate are those satisfiable by finite models. The non-termination is therefore pervasive, not limited to a single pathological example.

Remark 4.3 (Why strengthening the blocking condition does not help)**.** The frontier asymmetry arises because edges are assigned permanently at creation time: the new node gets PPI-edges to all existing (predecessor) nodes but has no PP-edges (successors do not yet exist). Any blocking condition that uses $\mathsf{Tri}(x)$ inherits this problem—the frontier's Tri is always strictly smaller. Strengthening the blocking condition (e.g., adding TNbr) makes non-termination *worse*, not better: more distinctions = less blocking = more unbounded creation.

5 Soundness

Theorem 5.1 (Soundness — **formal gap**)**.** *If the tableau produces an open completion graph $\mathcal{G}$ for C_0, then C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$.*

Gap. The proof depends on Lemma 5.8 (non-empty domains after arc-consistency), which is supported by computational verification but not by a complete formal argument. See Section 8.5, item 1.

The proof constructs a model directly from $\mathcal{G}$ via tree unraveling and triangle-type-filtered constraint solving.

5.1 Tree unraveling

Definition 5.2 (Tree unraveling)**.** Let $V_a \subseteq V$ be the active nodes. For each blocked node x, let $\beta(x)$ denote its blocker. The tree unraveling $(\Delta, \mathsf{map}, \mathsf{par}, \sigma)$ is defined inductively:

1. The root $d_0 \in \Delta$ has $\mathsf{map}(d_0) = x_0$.
2. For each $d \in \Delta$ with $\mathsf{map}(d) = n$, and each existential demand $\exists S.D \in \mathcal{L}(n)$ witnessed by node w (i.e., $\mathcal{E}(n, w) = S$ and $D \in \mathcal{L}(w)$): create a child d' with $\mathsf{par}(d') = d$, $\sigma(d') = S$, and
$$\mathsf{map}(d') = \begin{cases} w & \text{if } w \in V_a, \\ \beta(w) & \text{if } w \text{ is blocked.} \end{cases}$$

Each element's type is $\mathsf{tp}(d) = \mathcal{L}(\mathsf{map}(d))$.

Remark 5.3. When a blocked node w is redirected to its blocker $\beta(w)$, the child d' maps to $\beta(w)$ and inherits $\beta(w)$'s label. Since $\mathcal{L}(w) = \mathcal{L}(\beta(w))$ (condition (i) of blocking), the type is preserved. The critical property is that $\beta(w)$'s existential demands, when recursively expanded, produce the *same abstract triangle types* as w would have—guaranteed by conditions (ii) and (iii) of blocking: $\mathsf{Tri}(w) = \mathsf{Tri}(\beta(w))$ and $\mathsf{TNbr}(w) = \mathsf{TNbr}(\beta(w))$. The Tri-neighborhood condition further ensures that each witness v of $\beta(w)$ at relation R of type τ has a matching Tri-value to the corresponding witness of w, so the copy is faithful from every participant's perspective.

5.2 The triangle-type-filtered constraint network

Definition 5.4 (Type-safe relations)**.** For Hintikka types $\tau_1, \tau_2 \in \mathsf{Tp}(C_0)$ and $R \in \mathsf{NR}^-$, the relation R is *type-safe* for (τ_1, τ_2) if:

(a) $\forall R.D \in \tau_1 \Rightarrow D \in \tau_2$, and
(b) $\forall \mathsf{inv}(R).D \in \tau_2 \Rightarrow D \in \tau_1$.

We write $\mathsf{Safe}(\tau_1, \tau_2) = \{R \in \mathsf{NR}^- : R \text{ is type-safe for } (\tau_1, \tau_2)\}$.

Remark 5.5. If nodes a_1, a_2 in a saturated completion graph have $\mathcal{L}(a_1) = \tau_1$, $\mathcal{L}(a_2) = \tau_2$, and $\mathcal{E}(a_1, a_2) = R$, then $R \in \mathsf{Safe}(\tau_1, \tau_2)$ (by saturation of the $(\forall)$-rule). Thus $P(\mathcal{G}) \subseteq \{(\tau_1, R, \tau_2) : R \in \mathsf{Safe}(\tau_1, \tau_2)\}$. In particular, Safe is at least as permissive as $P(\mathcal{G})$.

Definition 5.6 (Constraint network $\mathcal{N}_T$)**.** Let $T = T(\mathcal{G})$ and $P = P(\mathcal{G}) = \{(\mathcal{L}(x), \mathcal{E}(x, y), \mathcal{L}(y)) : x \neq y \in V\}$. For each pair of distinct elements $d_1, d_2 \in \Delta$:

Parent–child edges: If $\mathsf{par}(d_2) = d_1$, then $\rho(d_1, d_2) = \sigma(d_2)$ (fixed).

Non-adjacent pairs: The initial domain is

$$D_0(d_1, d_2) \;=\; \{R \in \mathsf{NR}^- : (\mathsf{tp}(d_1), R, \mathsf{tp}(d_2)) \in P\} \;\cup\; \mathsf{Safe}(\mathsf{tp}(d_1), \mathsf{tp}(d_2)).$$

The first component draws on realized pair-types in $\mathcal{G}$; the second ensures that same-type pairs (including same-map pairs where $\mathsf{map}(d_1) = \mathsf{map}(d_2)$) always receive a non-empty initial domain whenever type-safe relations exist.

The *triangle-filtered constraint network* $\mathcal{N}_T$ is obtained by enforcing arc-consistency: iteratively remove any $R \in D(d_1, d_2)$ for which there exists d_3 such that for some $R_{23} \in D(d_2, d_3)$, no $R_{13} \in D(d_1, d_3)$ satisfies $(\mathsf{tp}(d_1), R, \mathsf{tp}(d_2), R_{23}, \mathsf{tp}(d_3), R_{13}) \in T$.

Remark 5.7 (Same-map pairs and unique types)**.** A reviewer [11] provided a concrete example showing that the $P(\mathcal{G})$-only domain can be empty for same-map pairs: if $d_1 \neq d_2$ both map to the sole node of type τ in $\mathcal{G}$, then $P(\mathcal{G})$ contains no triple (τ, R, τ) (edges exist only between distinct nodes). The Safe component resolves this: it includes every R that is type-safe for (τ, τ), without requiring a witness pair in $\mathcal{G}$.

In the exceptional case $\mathsf{Safe}(\tau, \tau) = \emptyset$—which occurs only when τ contains $\forall R.D$ with $D \notin \tau$ for every base relation R—any model has at most one element of type τ. The unraveling should then *identify* same-map copies of τ-nodes rather than creating distinct copies, producing a DAG domain in which same-map non-adjacent τ-pairs do not arise.

5.3 The key lemma: non-empty domains

Lemma 5.8 (Non-empty domains — **not formally proved**)**.** *Let $\mathcal{G}$ be an open completion graph with Tri-neighborhood blocking. Then the T-filtered constraint network $\mathcal{N}_T$ has non-empty domains for all pairs after arc-consistency.*

Status. The argument below establishes initial non-emptiness and explains the mechanism by which arc-consistency preserves non-emptiness (mirror-symmetry breaking via disjunctive domains). However, the "non-emptiness after filtering" step is verified computationally, not proved for arbitrary open completion graphs. This is the formal gap in the soundness proof.

Proof. We show that every domain $D_0(d_1, d_2)$ is initially non-empty, and that T-filtering combined with arc-consistency preserves non-emptiness.

Initial non-emptiness. For parent–child pairs, the relation is fixed by σ and corresponds to a witnessed existential demand. For non-adjacent pairs with distinct maps $\mathsf{map}(d_1) \neq \mathsf{map}(d_2)$: since $\mathcal{G}$ is a complete graph, the edge $\mathcal{E}(\mathsf{map}(d_1), \mathsf{map}(d_2))$ provides at least one element of $P(\mathcal{G})$, so $D_0 \neq \emptyset$. For same-map pairs ($\mathsf{map}(d_1) = \mathsf{map}(d_2) = n$): $P(\mathcal{G})$ may contribute nothing (if n is the sole node of type $\mathcal{L}(n)$), but $\mathsf{Safe}(\mathcal{L}(n), \mathcal{L}(n))$ provides all type-safe self-relations. When $\mathsf{Safe}(\mathcal{L}(n), \mathcal{L}(n)) \neq \emptyset$, $D_0 \neq \emptyset$. When $\mathsf{Safe}(\mathcal{L}(n), \mathcal{L}(n)) = \emptyset$, the unraveling identifies copies (Remark 5.7), so no same-map non-adjacent pair arises.

Why not a map-based solution? A natural approach would construct a *T-closed solution* (an assignment where every triple's triangle type is in T) by setting $\rho(d_1, d_2) = \mathcal{E}(n_1, n_2)$ where $n_i = \mathsf{map}(d_i)$. This works when all three maps are distinct: (n_1, n_2, n_3) is a triple in $\mathcal{G}$, so the triangle type is in $T(\mathcal{G})$. The Tri-neighborhood blocking conditions (ii) and (iii) ensure this extends correctly through blocked copies.

However, when two elements $d_1 \neq d_2$ both map to the *same* node n (a "same-map pair"), the map-based assignment fails. These pairs arise when the unraveling creates multiple copies of

a blocked subtree. The difficulty is that $\mathcal{E}(n,n)$ is undefined, and moreover, assigning *any* relation R to the pair creates *mirror triangles*: for a third element d_3 with $\mathsf{map}(d_3) = m \neq n$, both d_1 and d_2 would receive the same relation $\mathcal{E}(n,m)$ to d_3, producing the triangle type $(\mathcal{L}(n), R, \mathcal{L}(n), \mathcal{E}(n,m), \mathcal{L}(m), \mathcal{E}(n,m))$. Such mirror types may not be in $T(\mathcal{G})$: for instance, $(\tau_A, R, \tau_A, \mathsf{PO}, \sigma, \mathsf{PO})$ requires two distinct τ_A nodes both PO-related to the same σ witness, which never occurs in the PO-incoherent model. This obstruction is *structural*—no choice of R avoids it.

Resolution: disjunctive domains. The constraint network $\mathcal{N}_T$ does not commit to a single relation per pair. Instead, each non-adjacent pair carries the full disjunctive domain D_0. The T-filtering and arc-consistency steps narrow these domains while maintaining non-emptiness. The key mechanism:

Breaking mirror symmetry. For a same-map pair (d_1, d_2) both mapping to n, and a third element d_3 mapping to m with $\mathcal{E}(n,m) = \mathsf{PO}$: the domains $D(d_1, d_3)$ and $D(d_2, d_3)$ need not contain PO. The initial domain $D_0(d_i, d_3)$ includes all relations R such that $(\mathcal{L}(n), R, \mathcal{L}(m)) \in P$—typically $\{\mathsf{DR}, \mathsf{PO}, \mathsf{PPI}\}$ for the (τ_A, σ) pair. Arc-consistency may narrow this to $\{\mathsf{DR}, \mathsf{PPI}\}$ (removing PO from one of the edges), which eliminates the mirror triangle while retaining support from non-mirror triangle types that *are* in T.

Type-safety of the relaxation. Every relation in D_0 is type-safe by construction: relations from $P(\mathcal{G})$ are type-safe because the $(\forall)$-rule has saturated the corresponding pair in $\mathcal{G}$; relations from $\mathsf{Safe}(\mathsf{tp}(d_1), \mathsf{tp}(d_2))$ are type-safe by definition. Replacing PO with DR or PPI on a cross-subtree edge therefore satisfies all $\forall$-constraints—the new relation is *less demanding*, not more.

Non-emptiness after filtering. For each pair (d_1, d_2) and each $R \in D_0(d_1, d_2)$, R survives the combined T-filtering and arc-consistency if for every third element d_3, there exist $S \in D(d_2, d_3)$, $S' \in D(d_1, d_3)$ with $(\mathsf{tp}(d_1), R, \mathsf{tp}(d_2), S, \mathsf{tp}(d_3), S') \in T$. The disjunctive domains provide this flexibility: even when the map-based values $\mathcal{E}(n_1, n_3)$ and $\mathcal{E}(n_2, n_3)$ create mirror types, alternative values from D_0 participate in non-mirror types that are in T.

Computational verification (`intra_subtree_tclosure_check.py`) confirms: for completion graphs of size 8, 10, and 12 nodes, all domains remain non-empty after T-filtering and arc-consistency, despite the map-based assignment not being T-closed. The mirror PO is consistently replaced by DR or PPI on cross-subtree edges, breaking the symmetry while maintaining composition consistency. □

Remark 5.9 (The mirror triangle obstruction)**.** The mirror triangle obstruction is specific to the PO relation and the PO-incoherent case. It arises because PO is a "unique-witness" relation: each σ node w_k is PO-related to exactly one τ_A node d_{2k} and PPI/DR to all others. Two copies of d_{2k} would both need PO to w_k under the map, but the completion graph never contains two distinct τ_A nodes both PO-related to the same σ node.

The disjunctive constraint network resolves this precisely because it does not force the same relation on both copies. Instead, at most one copy retains PO to w_k; the other is assigned DR or PPI, which are valid for the (τ_A, σ) pair-type and participate in triangle types present in $T(\mathcal{G})$.

Remark 5.10 (Why weaker blocking conditions are insufficient)**.** With *type-equality blocking* alone ($\mathcal{L}(x) = \mathcal{L}(y)$, no conditions on Tri or TNbr), the blocker y and blocked node x may participate in *different* triangle types. Copying y's expansion for x can produce triples not in T, failing T-closure.

With *basic triangle-type-set blocking* ($\mathcal{L}(x) = \mathcal{L}(y)$ and $\mathsf{Tri}(x) = \mathsf{Tri}(y)$, but no condition on TNbr), the copy is correct from x/y's perspective, but not necessarily from a witness's perspective. When y's witness v is placed in x's context, triangles viewed from v's position may involve

neighbors of x whose own Tri-values differ from those of y's neighbors. The Tri-neighborhood condition (iii) closes this gap: every witness of $\beta(w)$ has a Tri-matching counterpart among w's neighbors.

5.4 From non-empty domains to model

Lemma 5.11 (Path-consistency)**.** *The arc-consistent fixpoint of $\mathcal{N}_T$ is path-consistent.*

Proof. By Lemma 5.8, all domains are non-empty after arc-consistency. By definition of arc-consistency: for every $R \in D(d_1, d_2)$ and every d_3, there exists $R_{23} \in D(d_2, d_3)$ and $R_{13} \in D(d_1, d_3)$ with $(\mathsf{tp}(d_1), R, \mathsf{tp}(d_2), R_{23}, \mathsf{tp}(d_3), R_{13}) \in T$. Since $T \subseteq \mathsf{Tp}^3 \times (\mathsf{NR}^-)^3$ records only composition-consistent triples (every triple in the completion graph satisfies the RCC5 composition table), this implies $\mathsf{comp}(R, R_{23}) \cap D(d_1, d_3) \neq \emptyset$, which is path-consistency. □

Theorem 5.12 (Model construction)**.** *Given an open completion graph $\mathcal{G}$ with Tri-neighborhood blocking, a model $\mathcal{I} \models C_0$ exists.*

Proof. By Lemma 5.8, the arc-consistent fixpoint of $\mathcal{N}_T$ has non-empty domains. By Lemma 5.11, it is path-consistent. By Theorem 2.2 (full RCC5 tractability), a consistent atomic refinement $\rho^{\mathcal{I}}$ exists.

Define $\mathcal{I} = (\Delta, \cdot^{\mathcal{I}})$ with $\rho^{\mathcal{I}}$ as the relation function. We verify:

Composition consistency. $\rho^{\mathcal{I}}$ is globally consistent by construction.

$\forall$-safety. For any non-adjacent edge $\rho^{\mathcal{I}}(d_1, d_2) = S$: since $S \in D_0(d_1, d_2)$ and every element of D_0 is type-safe (either witnessed in $P(\mathcal{G})$ where $(\forall)$ has saturated, or in $\mathsf{Safe}(\mathsf{tp}(d_1), \mathsf{tp}(d_2))$ by definition), we have $\forall S.D \in \mathsf{tp}(d_1) \Rightarrow D \in \mathsf{tp}(d_2)$. For parent–child edges, $\forall$-safety follows from tableau saturation.

$\exists$-satisfaction. For every d and $\exists S.D \in \mathsf{tp}(d)$: by construction, d has a child d' with $\sigma(d') = S$ and $D \in \mathsf{tp}(d')$.

Root. $C_0 \in \mathsf{tp}(d_0)$. □

6 Completeness

Theorem 6.1 (Completeness)**.** *If C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$, then there exists a sequence of nondeterministic choices yielding an open completion graph.*

Proof. Let $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ be a model with $d_0 \in C_0^{\mathcal{I}}$. We maintain a guide function $\pi : V \to \Delta^{\mathcal{I}}$ with invariants:

(I) $\mathcal{L}(x) \subseteq \mathsf{tp}^{\mathcal{I}}(\pi(x))$.
(II) $\mathcal{E}(x, y) = \rho^{\mathcal{I}}(\pi(x), \pi(y))$ for distinct x, y.

Initialization. $\pi(x_0) = d_0$. Invariant (I) holds since $\mathcal{L}(x_0) = \{C_0\}$ and $d_0 \in C_0^{\mathcal{I}}$.

Guiding $(\sqcup)$**.** When $D_1 \sqcup D_2 \in \mathcal{L}(x)$, invariant (I) gives $\pi(x) \in (D_1 \sqcup D_2)^{\mathcal{I}}$; choose the true disjunct.

Guiding ($\exists$). When $\exists R.D \in \mathcal{L}(x)$ with x active, $\pi(x) \in (\exists R.D)^{\mathcal{I}}$ gives $d' \in \Delta^{\mathcal{I}}$ with $\rho^{\mathcal{I}}(\pi(x), d') = R$ and $d' \in D^{\mathcal{I}}$. Create y, set $\pi(y) = d'$. For each z, choose $S_z = \rho^{\mathcal{I}}(\pi(z), d')$. Both invariants are maintained. Composition consistency (CF) holds because the model satisfies the composition table. After ($\forall$)-propagation, invariant (I) is maintained.

Clash-freeness. Invariant (I) prevents concept clashes; invariant (II) ensures composition consistency.

Demand satisfaction. The only concern is blocked nodes. Let x be blocked by y with $\mathcal{L}(x) = \mathcal{L}(y)$, $\mathsf{Tri}(x) = \mathsf{Tri}(y)$, and $\mathsf{TNbr}(x) = \mathsf{TNbr}(y)$. Suppose $\exists R.D \in \mathcal{L}(x)$. Then $\exists R.D \in \mathcal{L}(y)$. Since y is active and $\mathcal{G}$ is saturated, there exists w with $\mathcal{E}(y, w) = R$ and $D \in \mathcal{L}(w)$.

In the *model being constructed* (not the completion graph), x's demands are satisfied by the tree unraveling: x's blocked subtree is expanded using y's witnesses. The soundness theorem (Section 5) handles the model construction from the completion graph.

The guide function shows a sequence of consistent nondeterministic choices exists, yielding an open graph. □

7 Computational Evidence for Triangle-Type-Set Stabilization

The non-termination result (Theorem 4.1) shows that the calculus does not yield a decision procedure. Nevertheless, the computational evidence for triangle-type-set stabilization among *interior* nodes remains relevant, as it informs potential fixes to the blocking criterion.

7.1 The PO-incoherent counterexample

The hardest known case for blocking is the PO-incoherent descriptor from [7]: types τ_A, τ_B alternating on a PP-chain, with $\exists \mathsf{PO}.A \in \tau_A$ and $\forall \mathsf{PO}.\neg A \in \tau_B$. Each τ_A-node d_{2k} needs a fresh PO-witness w_k (earlier witnesses have become PPI-related).

7.2 Node-identity profiles never match

With node-identity profiling, each d_{2k} has a unique relational signature: PO to w_k, DR to $w_0, \ldots, w_{k-1}$, PPI to $w_{k+1}, w_{k+2}, \ldots$. Since k grows without bound, no two τ_A-nodes ever share a profile. This was verified computationally [8].

7.3 Abstract triangle-type sets DO match

Replacing node identities with abstract triangle types changes the picture entirely. The script `triangle_type_saturation_check.py` builds a 24-element PP-chain with 12 PO-witnesses and computes $\mathsf{Tri}(x)$ for every node x. Basic Tri-matching stabilizes at $k = 2$:

Node type	Tri stabilizes	Interior range	$\lvert\mathsf{Tri}\rvert$
τ_A	d_4 (k=2)	$d_4 = d_6 = \cdots = d_{18}$	68
τ_B	d_5 (k=2)	$d_5 = d_7 = \cdots = d_{19}$	56
σ	w_2 (k=2)	$w_2 = w_3 = \cdots = w_9$	57

7.4 Tri-neighborhood equivalence also stabilizes

The strengthened condition $\mathsf{TNbr}(x) = \mathsf{TNbr}(y)$ requires that the Tri-values among neighbors also match per (relation, type) pair. The script `tri_neighborhood_check.py` verifies this. Result: Tri-neighborhood equivalence stabilizes one step later, at $k = 3$:

Node type	Tri stabilizes	TNbr stabilizes
τ_A	d_4 (k=2)	d_6 (k=3)
τ_B	d_5 (k=2)	d_7 (k=3)
σ	w_2 (k=2)	w_3 (k=3)

The one-step delay has a clean explanation: at $k = 2$, a node's Tri-set has stabilized, but some of its neighbors are still in the start-boundary transient (e.g., d_4's PPI-neighbors include d_2 and w_1, which are boundary nodes with non-stabilized Tri-sets). At $k = 3$, *all* neighbors are also in the stabilized interior, so the TNbr signatures match.

The boundary nodes (d_0, d_2 at the start; d_{20}, d_{22} at the end) differ due to having fewer neighbors—the start boundary is a transient, the end boundary is a finite-model artifact absent from the infinite tableau construction. The result holds for both the all-DR-backward and all-PP-backward branches.

7.5 Why stabilization occurs

The abstract triangle-type set depends on the *menu* of (type, relation) pairs available among neighbors—not on the *count* of neighbors of each kind. Once a node has at least one predecessor of each relevant (type, relation) combination and at least one successor of each relevant combination, its menu is complete. Additional neighbors of the same abstract kind contribute no new triangle types.

For the TNbr condition, one additional step is needed: all neighbors must themselves have stabilized Tri-sets. Since all three node types (τ_A, τ_B, σ) stabilize simultaneously at $k = 2$, a node at depth $k = 3$ has all its neighbors at depth ≥ 2, hence all with stabilized Tri-sets.

8 Discussion

8.1 The blocking dilemma—confirmed

The counterexample of Section 4.1 confirms that the blocking dilemma is genuine:

Blocking condition	Terminates?	Correct unraveling?
Type-equality ($\mathcal{L}$ only)	Always	Not always
Node-identity profile	Not always	Always
Triangle-type-set ($\mathcal{L} + \mathsf{Tri}$)	**Not always**	Yes
Tri-neighborhood ($\mathcal{L} + \mathsf{Tri} + \mathsf{TNbr}$)	**Not always**	**Yes**

No blocking condition based on Tri can resolve the dilemma: the frontier-advancement mechanism (Remark 4.3) affects *every* condition that compares Tri-values, because the frontier node's Tri

is inherently smaller than any interior node's. Weakening to type-equality recovers termination but loses the triangle-type-preservation guarantee needed for soundness.

The key insight—that abstract relational patterns (not concrete node identities) are the right level of abstraction for both blocking and unraveling—remains valid. What is needed is a mechanism that ensures the frontier node's Tri matches interior nodes *despite* the asymmetry in PP/PPI edges. Possible directions include:

- *Lazy blocking*: defer the Tri comparison until the frontier node acquires successors.
- *Prophetic Tri*: compute Tri assuming the frontier's successors will mirror the blocker's.
- *Hybrid approach*: use type-equality for termination, then verify soundness by checking that the completed graph's triangle types support unraveling.

We leave these directions as open problems.

8.2 Relationship to the two-tier quotient

The two-tier quotient approach [7] proves decidability for the PO-coherent fragment via a model-theoretic argument. The tableau approach here aims to handle the full logic, including PO-incoherent descriptors. The two approaches are complementary:

- The quotient approach gives a cleaner decidability proof for the PO-coherent fragment.
- The tableau approach addresses the PO gap by using Tri-neighborhood blocking, which is insensitive to the PO-coherent/incoherent distinction.

8.3 Complexity

Since the calculus does not terminate, complexity analysis is moot for the current blocking condition. The active-node count is bounded (triply exponential in $|C_0|$), but total node creation is unbounded. A terminating variant—if one can be found—would need separate complexity analysis.

8.4 Extension to $\mathcal{ALCI}_{RCC8}$

RCC8 has 8 base relations and a more complex composition table. The patchwork property fails for the full RCC8, but holds for the maximal tractable subsets $\hat{\mathcal{H}}_8$ identified by Renz [3]. The tableau approach extends to $\mathcal{ALCI}_{RCC8}$ if the pair-type domains $P(\mathcal{G})$ fall within a tractable fragment. This is a non-trivial condition that we do not address here.

8.5 Honest assessment of gaps

We identify the following points where the proof requires further scrutiny:

1. **Intra-subtree T-closure (Lemma 5.8).** The proof that arc-consistency preserves non-empty domains is supported by computational verification on completion graphs of size 8, 10, and 12, but not by a general formal argument. The mirror-symmetry-breaking mechanism (replacing PO with DR or PPI on cross-subtree edges) works in all tested cases, and every relation in D_0 is type-safe by construction. A fully formal proof that domains remain non-empty after T-filtering and arc-consistency for *arbitrary* open completion graphs would close this gap.

2. **Initial domains for same-map pairs.** An earlier version defined D_0 using only $P(\mathcal{G})$ (realized pair-types between distinct completion-graph nodes). As observed by a reviewer [11], this gives $D_0 = \emptyset$ when two unraveling elements both map to the sole node of a given type. The current version resolves this by extending D_0 with $\mathsf{Safe}(\tau_1, \tau_2)$ (Definition 5.4), which provides type-safe self-relations without requiring a witness pair in $\mathcal{G}$. For the degenerate case $\mathsf{Safe}(\tau, \tau) = \emptyset$ (which forces at most one model element of type τ), the unraveling identifies same-map copies rather than creating distinct elements (Remark 5.7).

3. **Termination (Theorem 4.1).** The termination conjecture from the second revision is **false**. A full implementation of the calculus (Section 4.1) shows unbounded node creation on the concept $A \sqcap \exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.(\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B)$. The mechanism is frontier advancement, not oscillation. The local per-node bounds (bounded branching, permanent demand satisfaction, monotone Tri-growth) are all individually correct but insufficient: the frontier node always escapes blocking because its Tri is strictly smaller than any interior node's.

   Two reviewers [10, 11] correctly identified the missing global argument. The counterexample vindicates their concern: the gap is not merely a proof difficulty but reflects genuine non-termination.

4. **Stabilization depth.** The earlier computational evidence showing Tri stabilization at $k = 2$ and TNbr stabilization at $k = 3$ remains correct *for interior nodes*. The issue is that stabilization of interior nodes is insufficient: the frontier node never stabilizes because it never acquires PP-successors before its $\exists$-demands fire.

9 Conclusion

We have presented a tableau calculus for $\mathcal{ALCI}_{RCC5}$ with *Tri-neighborhood blocking*: a node x is blocked by an earlier node y when (i) $\mathcal{L}(x) = \mathcal{L}(y)$, (ii) $\mathsf{Tri}(x) = \mathsf{Tri}(y)$, and (iii) $\mathsf{TNbr}(x) = \mathsf{TNbr}(y)$. The calculus is *complete* (a model guides all nondeterministic choices). The soundness argument (model construction via triangle-type-filtered unraveling, with the Safe extension for same-map pairs and global consistency from full RCC5 tractability) has a **formal gap**: non-empty domains after arc-consistency (Lemma 5.8) is supported by computational verification but not by a general formal proof.

Termination also fails. The concept $A \sqcap \exists\mathsf{PP}.A \sqcap \forall\mathsf{PP}.(\exists\mathsf{PP}.A \sqcap \exists\mathsf{DR}.B)$ causes unbounded node creation via a frontier-advancement mechanism: the newest chain node always has $|\mathsf{Tri}| = 8$ (only PPI-predecessor triangles), while interior nodes have $|\mathsf{Tri}| = 16$ (both PP and PPI triangles), so the frontier is never blocked. This confirms the *blocking dilemma*: type-equality blocking terminates but risks unsound unraveling; Tri-based blocking ensures sound unraveling but does not terminate.

The paper should therefore be read as *partial progress* toward the decidability of $\mathcal{ALCI}_{RCC5}$: only completeness is fully established; soundness has a formal gap and the calculus does not terminate. The main contributions are: (1) the Tri-neighborhood blocking condition and the argument that it ensures correct unraveling (contingent on Lemma 5.8); (2) the disjunctive constraint-network approach to same-map pairs, with the Safe extension; (3) computational evidence confirming the frontier-advancement mechanism as the precise obstacle to termination; and (4) three conjectured directions for resolving the dilemma (lazy blocking, prophetic Tri, hybrid approaches).

The decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ remains open.

References

[1] M. Wessel. Qualitative spatial reasoning with the $\mathcal{ALCI}_{RCC}$ family — first results and unanswered questions. Technical Report FBI-HH-M-324/03, University of Hamburg, 2002/2003.

[2] J. Renz and B. Nebel. On the complexity of qualitative spatial reasoning: a maximal tractable fragment of the region connection calculus. *Artificial Intelligence*, 108(1-2):69–123, 1999.

[3] J. Renz. Maximal tractable fragments of the region connection calculus: a complete analysis. In *IJCAI*, 1999.

[4] Claude (Anthropic) and M. Wessel. On the decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$. Unpublished manuscript, 2026.

[5] GPT-5.4 (OpenAI). Contextual tableau for $\mathcal{ALCI}_{RCC5}$: soundness via local-state unfolding. Unpublished manuscript, 2026.

[6] Claude (Anthropic) and M. Wessel. Closing the extension gap for $\mathcal{ALCI}_{RCC5}$. Unpublished manuscript, 2026. **Retracted.**

[7] Claude (Anthropic) and M. Wessel. Decidability of $\mathcal{ALCI}_{RCC5}$ via two-tier quotient construction. Unpublished manuscript, fourth revision, April 2026.

[8] `profile_blocking_check.py`, `triangle_type_saturation_check.py`, `tri_neighborhood_check.py`, and `intra_subtree_tclosure_check.py`. Computational verification scripts, April 2026. Available at https://github.com/lambdamikel/alcircc5.

[9] `triangle_calculus.py` and `triangle_nonterm.py`. Full tableau implementation and non-termination analysis, April 2026. Available at https://github.com/lambdamikel/alcircc5.

[10] GPT-5.4 Pro (OpenAI). Technical response to "A tableau calculus for $\mathcal{ALCI}_{RCC5}$ with triangle-type-set blocking." First review memo, April 2026.

[11] GPT-5.4 Pro (OpenAI). Technical response to the second revision of "A tableau calculus for $\mathcal{ALCI}_{RCC5}$ with Tri-neighborhood blocking." Second review memo, April 2026.

Acknowledgments

This research was prompted by Michael Wessel, who introduced the $\mathcal{ALCI}_{RCC}$ family in his doctoral work at the University of Hamburg. The triangle-type-set blocking idea arose from Wessel's critical observation that abstract triangle-type sets—unlike node-identity profiles—should stabilize in the PO-incoherent case. The strengthened Tri-neighborhood condition was also suggested by Wessel, who observed that requiring matching Tri-values among neighbors (not just at the blocked/blocker pair) would close the intra-subtree T-closure gap in the soundness argument.